



**TECNOLOGIA
SETÚBAL**

ESCOLA SUPERIOR
POLITÉCNICO SETÚBAL

Ano letivo 2025/2026

2º Semestre

Programação Aplicada à Cibersegurança

Reverse Engineering e Scanning de Portas
Projeto

CTeSP em Cloud e Cibersegurança

Data: 15/06/2026

Pedro Mota - 2025126201

Jeremias Amado - 2025127142

Índice

<i>INTRODUÇÃO</i>	3
<i>Temas Escolhidos</i>	4
Tema 3 - Reverse Engineering	4
Tema 12 - Scanning de Portas e Serviços Mal Configurados	4
<i>Arquitetura do Projeto</i>	5
<i>Desenvolvimento</i>	5
Análise Estática Local	5
Comunicação via Sockets	6
Análise com Orquestrador	7
Scanning de Portas e Geração de Exploits	8
<i>Dificuldades</i>	8

INTRODUÇÃO

Este projeto foi desenvolvido no âmbito da unidade curricular Programação Aplicada à Cibersegurança II, do CTeSP em Cloud e Cibersegurança do Instituto Politécnico de Setúbal.

O objetivo principal foi criar um Orquestrador autónomo que combina dois temas: Reverse Engineering (Tema 3) e Scanning de portas e serviços mal configurados (Tema 12).

O Orquestrador é um script em Python que coordena, decide e executa tarefas de análise de malware e scanning de rede de forma automática. A solução integra análise estática local de ficheiros executáveis (extração de strings, deteção de packers e leitura do cabeçalho PE) com análise dinâmica remota, recorrendo ao Kali Linux através de comunicação por sockets. Quando a análise local não encontra informação suficiente como IPs ou portas o Orquestrador envia automaticamente o ficheiro para o Kali Linux remoto, onde é realizada uma análise mais aprofundada com a ferramenta Radare2.

Após a análise, o Orquestrador efetua um scan ao IP encontrado, identifica portas abertas e gera exploits específicos consoante as portas detetadas por exemplo, EternalBlue para a porta 445 ou brute force SSH para a porta 22.

A comunicação entre o PC do Jeremias e o Kali do Pedro é feita via sockets.

TEMAS ESCOLHIDOS

Tema 3 - Reverse Engineering

O Reverse Engineering (engenharia reversa) consiste na análise de software com o objetivo de compreender o seu funcionamento interno sem acesso ao código fonte original. No contexto da cibersegurança, esta técnica é utilizada para analisar malware, identificar vulnerabilidades e compreender comportamentos maliciosos.

Neste projeto, a análise de engenharia reversa foca-se em ficheiros PE (Portable Executable), extraíndo strings, detetando packers e interpretando o cabeçalho do executável para identificar indicadores de comprometimento.

Tema 12 - Scanning de Portas e Serviços Mal Configurados

O scanning de portas é uma técnica de reconhecimento que permite identificar portas abertas num sistema alvo, bem como os serviços que nelas correm. Serviços mal configurados ou desatualizados representam vetores de ataque frequentemente explorados por atacantes.

Neste projeto, o scanning é realizado após a análise de engenharia reversa, utilizando os IPs e portas encontrados para gerar exploits automaticamente com base nos serviços detetados.

```
-----  
FASE 1: RE LOCAL  
-----  
[*] A analisar...  
  
[*] Entry Point: 0x0  
[*] Packer: Possível packer detectado  
[*] DLLs: ['kernel32.dll', 'user32.dll', 'advapi32.dll']  
[*] Strings: 551450  
  
[*] Strings:  
-> <?xml version="1.0" encoding="UTF-8" standalone="yes"?>  
<assembly xmlns="urn:sch  
-> 'Í!  
-> Lí!This program cannot be run in DOS mode.  
$  
-> ÊktFÊktFÊktF³êqG|ktF³êpGÆktF³êwGÁktFCà  
-> FÊktFCàwGÃktFCàpGÜktFCàqGíktF³êuGíktFÊkuFFktF\æpG×ktF\ævGÊktFRichÊktF  
-> ÊÏ1j  
-> .text  
-> `.rdata
```

ARQUITETURA DO PROJETO

O projeto tem dois lados:

- PC do Jeremias: faz a análise estática local e comunica com o Kali via socket;
- Kali Linux do Pedro: recebe os comandos e ficheiros, analisa com Radare2 e devolve os resultados

O fluxo de funcionamento é o seguinte:

1. O Orquestrador recebe o ficheiro suspeito
2. Faz análise estática local como strings, packers e cabeçalho PE
3. Se não encontrar informação suficiente manda o ficheiro para o Kali via socket
4. O Kali analisa com Radare2 e devolve os resultados
5. O Orquestrador faz scan ao IP encontrado com nmap
6. Gera os exploits consoante as portas abertas modo inteligente

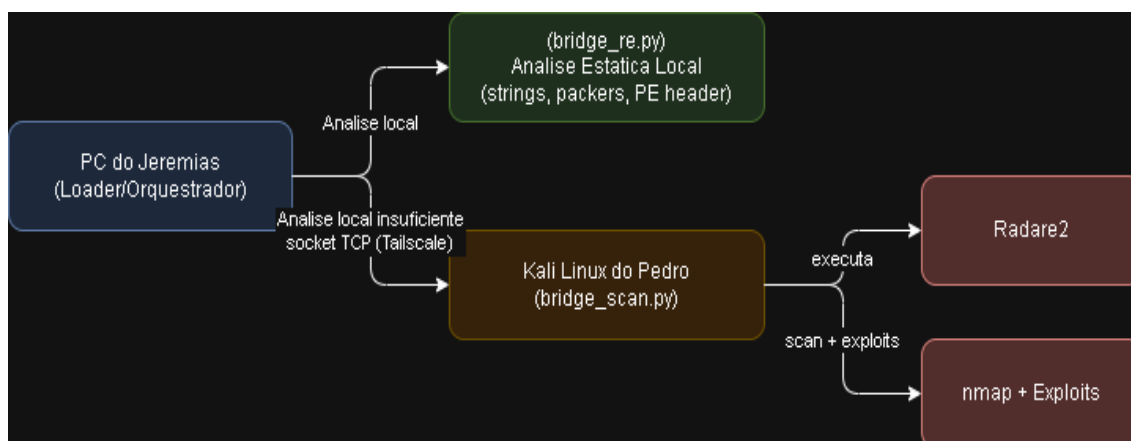


Figura 1 - Diagrama do projeto

DESENVOLVIMENTO

Análise Estática Local

A análise estática foi feita no ficheiro `bridge_re.py` sem usar bibliotecas externas, porque o professor pediu para mostrarmos que percebemos como o formato PE funciona.

Abrimos o ficheiro em modo binário com `open()` e verificamos os primeiros bytes. Se o ficheiro começar com `b'MZ'` e tiver a assinatura `bPE\x00\x00` no offset correto então é um executável PE válido. Para ler os offsets do cabeçalho usamos `int.from_bytes()`.

Para extrair *strings* percorremos o ficheiro byte a byte, aplicamos `decode('latin-1', errors=ignore)` e filtrámos com `isprintable()` para ficar só com texto legível. Para detetar

packers fizemos uma lista com nomes como *upx* e *vmprotect* e verificamos se algum aparecia nas *strings* extraídas.

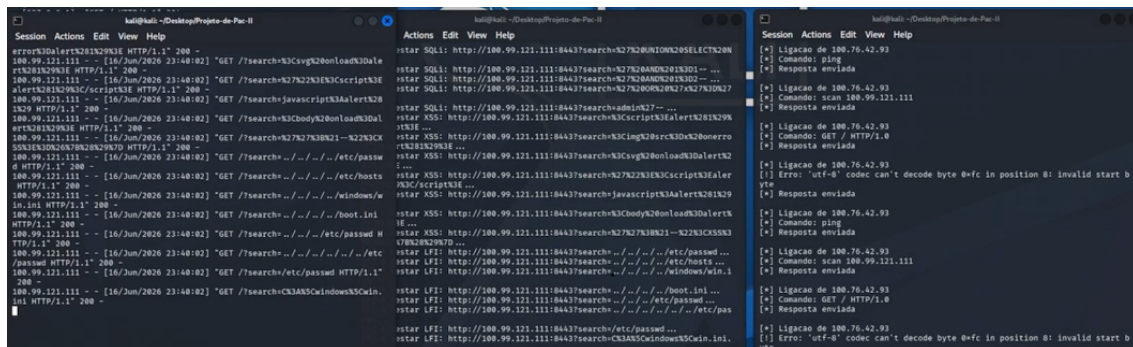
Comunicação via Sockets

A comunicação entre os nossos computadores é feita por sockets TCP. O loader envia comandos de texto e o Kali responde com os resultados.

Os comandos principais são:

- ping - O Kali responde pong para testar se a ligação está ativa
- scan 192.168.x.x - O Kali corre o nmap e devolve o output
- ENVIAR_FICHEIRO - O loader envia o malware em binário e recebe o resultado da análise
- gerar_exploit 192.168.x.x 445 - O Kali gera o exploit e devolve o caminho do ficheiro

Quando para enviar um ficheiro, o Orquestrador primeiro envia o tamanho do ficheiro, depois envia os dados em pedaços de 4096 bytes e por fim envia FIM para indicar que terminou.



```
netcat -l -p 4444
[+] Ligação de 100.76.42.93
[+] Comando: ping
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Comando: scan 100.99.121.111
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Comando: GET / HTTP/1.0
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Erro: 'utf-8' codec can't decode byte 0xfc in position 8: invalid start b
yte
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Comando: ping
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Comando: scan 100.99.121.111
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Comando: GET / HTTP/1.0
[+] Resposta enviada
[+] Ligação de 100.76.42.93
[+] Erro: 'utf-8' codec can't decode byte 0xfc in position 8: invalid start b
yte
```

Analise com Orquestrador

Quando a analise local nao é suficiente, o ficheiro é enviado para o Kali onde o orquestrador com bridge_scan.py encontra o ip e as portas todas automáticas com

A faz a analise automática de funções e referencias, e depois corre sem interaçao e o aflista todas as funções encontradas no binário tudo com base o que encontra nada fixado.

```
-----
FASE 2: PESCAR IPS COM CONTEXTO
-----

[*] IPs encontrados com contexto:
  1. 100.99.121.111 (tipo: KALI_DO_PEDRO)
     Contexto: nifestVersion="1.0">
<assemblyIdentity version="100.99.121.111" processorArchi...
  2. 6.0.0.0 (tipo: DESCONHECIDO)
     Contexto: name="Microsoft.Windows.Common-Controls" version="6.0.0.0" processorAr
chitecture...

-----
FASE 3: IDENTIFICAR IP DO PEDRO
-----

[*] Encontrei o Kali do Pedro: 100.99.121.111
[*] Queres entrar no Kali do Pedro? (s/n): s

-----
FASE 4: SCAN AO IP
-----

[*] A escanear 100.99.121.111...

[*] Portas abertas encontradas: 8080, 8443, 9999
```

Scanning de Portas e Geração de Exploits

Depois de ter o IP, o Orquestrador manda o comando "scan IP" para o Kali que corre:

`nmap -F -sV IP`

O -F faz um scan rápido nas portas mais comuns e o -sV detecta as versões dos serviços. Também temos a opção de detectar WAF com:

Com base nas portas abertas, o Orquestrador gera exploits automaticamente

```
-----  
FASE 6: TESTE WEB  
-----
```

```
[*] Testando http://100.99.121.111:8080  
[*] Resultado do teste web para 100.99.121.111:8080  
Nenhuma vulnerabilidade encontrada.  
[*] Credenciais: 100.99.121.111:8080  
  
[*] Testando http://100.99.121.111:8443  
[*] Resultado do teste web para 100.99.121.111:8443  
Nenhuma vulnerabilidade encontrada.  
[*] Credenciais: 100.99.121.111:8443
```

DIFICULDADES

A maior dificuldade que tivemos foi a comunicação entre os dois PCs porque estávamos em casas diferentes. Para resolver instalamos o *Tailscale* que cria uma rede virtual privada e assim o *loader* conseguia ligar-se ao Kali do Pedro com um IP fixo.

Outra dificuldade foi compilar o malware para .exe. Tentamos correr o PyInstaller na pasta C:\Windows\system32 e deu erro. Depois percebemos que tínhamos de estar na pasta do projeto e aí funcionou sem problemas.

O Radare2 às vezes demorava muito a analisar especialmente em binários maiores. Tivemos de aumentar o *timeout* no código para não dar erro antes de acabar a análise. Tentamos usar o *Ghidra* também mas era muito lento, chegava a demorar entre 2 e 5 minutos, por isso acabamos por não usar e ficamos só com o Radare2.

No início o *loader* também não detectava o Kali porque o IP do Pedro não estava definido. Resolvemos adicionando o IP manualmente e usando o *Tailscale* para garantir que os IPs eram sempre os mesmos entre as duas máquinas.

CONCLUSÃO

Neste projeto conseguimos criar um Orquestrador em *Python* que junta o *Reverse Engineering* com o *Scanning* de Portas numa só ferramenta. Achamos que foi um projeto interessante porque combina dois temas importantes na área de cibersegurança e faz tudo de forma automática.

Os objetivos que tínhamos definido foram alcançados. O Orquestrador consegue analisar um ficheiro de *malware* localmente, enviar para o Kali quando precisa de uma análise mais aprofundada com o Orquestrador, fazer scan às portas do IP encontrado e gerar exploits automaticamente com base nas portas abertas.

Se tivéssemos mais tempo melhorariamos o tempo de resposta do Orquestrado em binários maiores e tentariamos adicionar mais *exploits* para outras portas. Também gostaríamos de ter uma interface mais apresentável para mostrar os resultados em vez de ser só no terminal.